

Report Del : 13/01/2026

Studente Pancrazio Giuzio

Traccia:

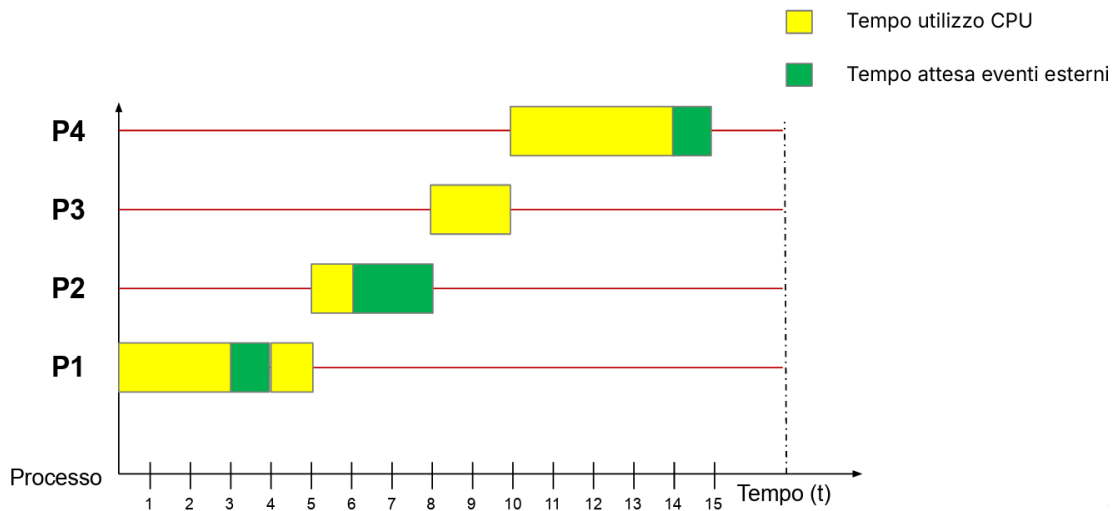
Si considerino 4 processi P1, P2, P3, P4) con i tempi di esecuzione e di attesa input/output dati in tabella. I processi arrivano alle CPU in ordine P1, P2, P3, P4. Individuare il modo più efficace per la gestione e l'esecuzione dei processi, tra i metodi già visti a lezione.

Abbozzare un diagramma che abbia sulle ascisse il tempo passato da un istante «0» e sulle ordinate il nome del Processo. Processo Tempo di esecuzione Tempo di attesa Tempo di esecuzione dopo

Processo	Tempo di esecuzione	Tempo di attesa	Tempo di esecuzione dopo attesa	
p1	3	1	1	5
p2	1	2	-	3
p3	2	-	-	2
p4	4	1	-	5

Svolgimento del report :

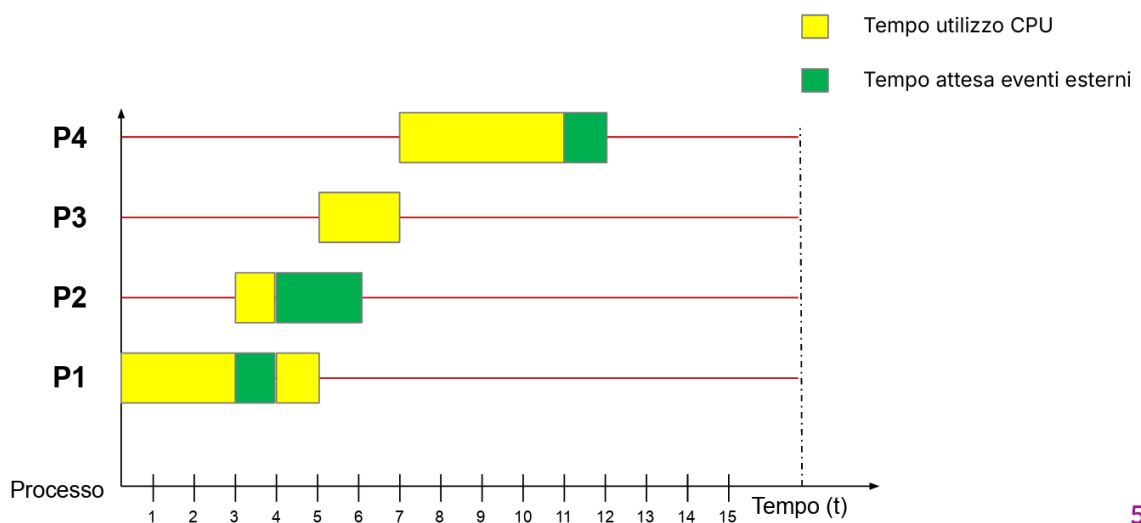
Si considera che i processi p1,p2,p3,p4 sono processi che posso passare attraverso un processore che puo' lavorare in tre modi : 1 In mono tasking (i processi vengono eseguiti uno alla volta fin quando non sono completati) questi sistema di elaborazione della cpu è piuttosto datato veniva usato soprattutto nei primi sistemi operativi come ms-dos da riga di comando. un grande svantaggio di questo sistema è il fatto che la cpu passa molto tempo compiendo cicli nei quali non sta lavorando. Qui l'illustrazione del tempo della cpu considerato che il tempo di attesa è fattore esterno . Andiamo quindi ad effettuare una semplice operazione di addizione in mono tasking da p1 a p4 considerando che i numeri in tabella sono secondi e che i tempi di attesa siano prestabiliti e non dati dall'interfaccia I/O utente : P1 =5 sec, P2 =3 sec, P3=2 sec, P4= 5 sec . $T(tot)=15sec$



2: Si prende in considerazione il sistema multi tasking , che utilizza la CPU per andare da un processo ad un altro durante i tempi di attesa I/O , in questo caso, in ogni tempo di attesa la CPU inizierà un altro processo

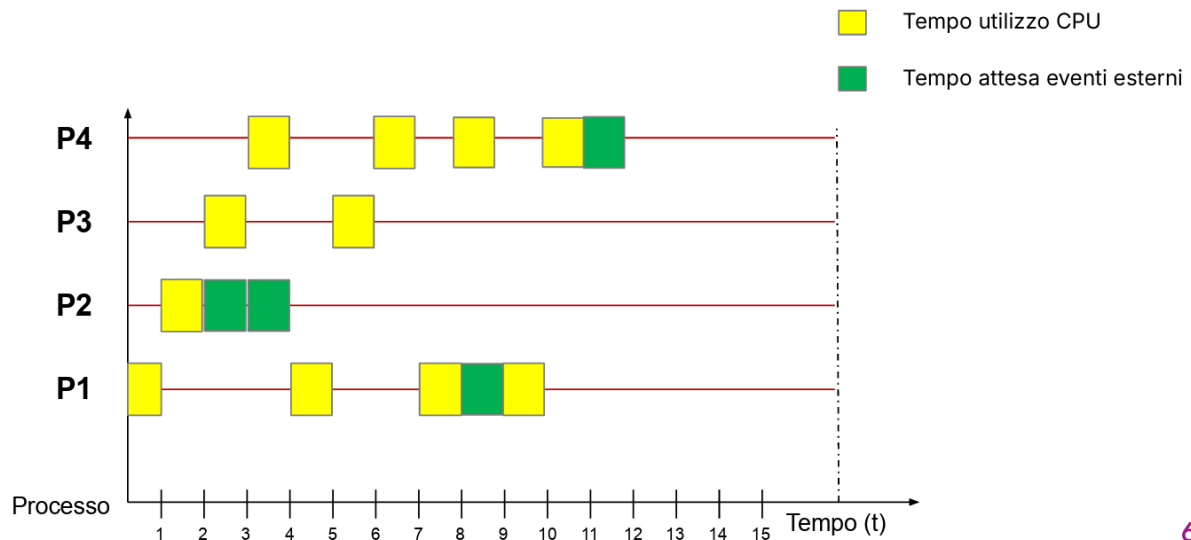
Processo	Tempo di esecuzione	Tempo di attesa	Tempo di esecuzione dopo attesa	
p1	3	1	1	
p2	1	2	-	
p3	2	-	-	
p4	4	1	-	

prendendo in considerazione questa tabella facciamo il semplice ragionamento che il tempo di attesa di p1 è proprio un secondo , la cpu in quel secondo andrà a impiegare le sue risorse per iniziare il p2 avendo così un totale sui tempi di attesa che scenderà nel tempo totale quindi: $T(\text{tot}) = 12 \text{ sec}$



3: si prende in considerazione il time shared tra i processi , dove si assegna un quanto ovvero un tempo di esecuzione pari a t tempo dato a tutti e 4 i processi da eseguire in modo da tenere per ogni processo un tempo condiviso anche con i processi non ancora completati.

Si prende in considerazione il time shared con 1 secondo di quanto . in questo caso il tempo totale $T(\text{tot}) = 12 \text{ sec}$



6

Conclusioni:

si può tranquillamente notare come il multitasking o il time sharing rispetto al mono task (attualmente quasi in disuso) abbiano una velocità maggiore in quanto non hanno bisogno di aspettare la fine di un processo per far lavorare la CPU su un altro, questo sicuramente dà un vantaggio per quanto riguarda processi che aspettano un input o che stiano aspettando ulteriore istruzione, ci consente allo stesso tempo di eseguire più operazioni su un PC e l'utilizzo di più risorse.